

E-Commerce Search & Recommendation System

A high-level, buildable project specification for a product discovery

platform with hybrid search and a personalized recommendation engine,

inspired by the curation philosophy of flash.co.

Project codename	ShopSense
Topic ID	E-Commerce Search and Recommendation System
Inspiration	flash.co (curated, evidence-ranked discovery)
Document type	Project Idea PRD (high level)
Version	1.0
Date	October 5, 2026
Audience	Builder / academic project team

Inspired by publicly observable features of flash.co. Not affiliated with or endorsed by Flash.co.

Table of Contents

1. Project Overview
2. Problem Statement & Motivation
3. Goals, Objectives & Scope
4. Inspiration Analysis: What to Borrow from flash.co
5. Target Users & Use Cases
6. Functional Requirements
7. Non-Functional Requirements
8. System Architecture
9. Search System Design
10. Recommendation System Design
11. Data Model
12. API Design
13. Technology Stack
14. Algorithms & Data Structures
15. Development Roadmap & Milestones
16. Testing, Evaluation & Metrics
17. Deployment, DevOps & Operations
18. Risks & Mitigations
19. Future Scope
20. Deliverables & Report Outline
21. Appendices (Datasets, sample payloads, glossary)

HOW TO READ THIS DOCUMENT

Sections 1-5 define the idea and scope. Sections 6-7 are the requirements. Sections 8-14 are the design: architecture, the two core engines (search and recommendation), data, API, stack and algorithms. Sections 15-20 cover delivery, evaluation, deployment, risks and deliverables. Sections 9 and 10 are the heart of the topic: read them first.

1. Project Overview

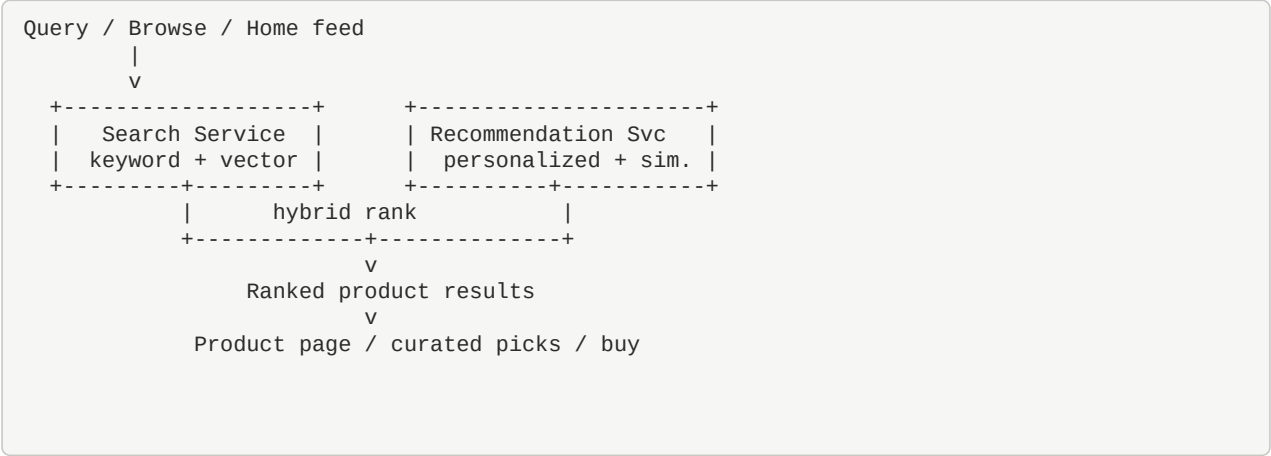
ShopSense is an e-commerce product discovery platform built around two engines: a Search engine that finds the most relevant products for an explicit query, and a Recommendation engine that proactively surfaces products a user is likely to want. Together they answer the two fundamental shopping questions: 'what am I looking for?' and 'what should I look at next?'.

The project takes its curation philosophy from flash.co: instead of dumping thousands of results, it aims to surface a small, high-quality, explainable set of picks. Where flash.co uses autonomous research agents to justify its picks, this project achieves a similar outcome with classical and modern information-retrieval and recommender techniques that are well suited to a single-semester build.

1.1 The two pillars

Pillar	Question it answers	Core techniques
Search	What do I want?	Inverted index, BM25, embeddings, hybrid fusion, filters, autocomplete
Recommendation	What should I consider next?	Popularity, content-based, item-item CF, matrix factorization, hybrid ranking

1.2 High-level user flow



1.3 Why this project is strong

- Covers a recognized, in-demand domain (search relevance + recommender systems).
- Naturally exercises core DSA (indexes, heaps, graphs, hashing, similarity) and system design.
- Has clear, measurable evaluation metrics (nDCG, MAP, Precision@K, Recall@K).
- Scales from a simple MVP to advanced ML without changing the architecture.
- The flash.co inspiration gives it a modern, differentiated product angle.

2. Problem Statement & Motivation

Modern e-commerce suffers from four problems that this project directly addresses.

Problem	Impact on users	What ShopSense does
Choice overload	Too many products; decision paralysis	Curation: rank and shortlist the best few
Weak search relevance	Irrelevant results; keyword-only matching misses intent	Hybrid lexical + semantic search with reranking
Generic recommendations	Same 'popular' items for everyone	Personalized + context-aware recommendations
Low trust	Ratings/reviews gamed; sponsored results	Transparent, explainable scoring; 'why recommended'

Motivation: as product catalogues grow, the bottleneck shifts from having products to finding the right one. Search and recommendation are the highest-leverage levers on conversion and satisfaction, which makes this topic both academically rich and commercially relevant.

3. Goals, Objectives & Scope

3.1 Objectives

- O1: Build a fast keyword search that ranks by relevance (BM25) and supports filters and facets.
- O2: Add semantic search so queries match meaning, not just exact words.
- O3: Fuse lexical and semantic signals into a single hybrid ranking.
- O4: Deliver personalized recommendations plus similar-item and trending recommendations.
- O5: Handle cold start (new users and new products) gracefully.
- O6: Make every ranking explainable and evaluate it with standard IR/RecSys metrics.

3.2 In scope

- Product catalogue with categories, brands and attributes.
- Hybrid search (keyword + semantic), filters, facets and autocomplete.
- Recommendation: trending, similar products, users-also-viewed, personalized feed.
- Explainable ranking/score on product and list pages.
- Curated 'top picks' lists inspired by flash.co.
- User accounts, interaction logging, wishlist and history.
- Admin dashboard for catalogue and basic operations.

3.3 Out of scope (v1)

- Real payments, order fulfilment and logistics.
- Autonomous LLM research agents and live large-scale web crawling.
- Multi-store live price comparison (a stretch goal in Section 19).
- Native mobile apps (responsive web first).

3.4 Assumptions

- Dataset is a curated public e-commerce dataset, optionally supplemented with compliant scraping.
- Web-first, mobile-responsive; single currency and locale.
- Solo builder or a small team with full-stack knowledge; one semester (~12-14 weeks).

4. Inspiration Analysis: What to Borrow from flash.co

flash.co is an AI-agent-curated shopping discovery site whose tagline is 'Not every product. Only the top 1%.' It ranks products from independent evidence and shows a transparent score, curated lists and live prices. We borrow the philosophy and the product patterns, not the operational complexity.

flash.co pattern	Borrow / Adapt / Skip	How ShopSense implements it
Curated ranked lists ('top 1%')	Borrow	Ranked lists with a transparent composite score
Transparent score per product	Borrow	Explainable ranking with visible contributing factors
Evidence-backed claims	Adapt	Score factors derived from ratings, reviews and attributes
Category browse + attribute clusters	Borrow	Faceted categories and generated 'best for' cuts
'Why recommended' explainability	Borrow	Human-readable reason on every recommendation
Live agent research pipeline	Skip	Replace with offline batch pipeline (feasible in a semester)
Affiliate multi-store price crawl	Skip (stretch)	Optional price comparison as future scope
Editorial content	Skip	Not core to the search/recommendation topic

5. Target Users & Use Cases

Persona	Need	Key features
Explorer	Browse and discover without a specific query	Home feed, trending, personalized recommendations
Intent-driven shopper	Find a specific product quickly	Keyword search, autocomplete, filters
Comparer	Evaluate alternatives	Similar products, ranked lists, score factors
Deal/quality seeker	Find the best value	Curated top picks, attribute filters
New user	Get useful results with no history	Cold-start recommendations, popularity

5.1 Representative use cases

1. A user types 'running shoes for flat feet' and gets relevant, filtered results even though exact words differ.
2. A user opens a product and sees similar and 'users also viewed' items.
3. A returning user sees a personalized home feed based on prior interactions.
4. A new user sees trending and category-best picks (cold start).
5. A user opens a curated list such as 'Best Wireless Earbuds under Rs.2,000' with ranked, scored items.

6. Functional Requirements

Priority: P0 = must-have for the project demo, P1 = should-have, P2 = stretch.

6.1 Catalogue and ingestion

1. FR-1 (P0) Catalogue: products with title, description, brand, category, price, images, attributes and ratings.
2. FR-2 (P0) Ingestion: bulk import from a dataset (CSV/JSON) and/or a connector; normalize and deduplicate.
3. FR-3 (P0) Taxonomy: hierarchical categories and brands; attribute schema per category.
4. FR-4 (P1) Index sync: products are indexed for search and made available to the recommender after ingestion.

6.2 Search

5. FR-5 (P0) Keyword search with relevance ranking (BM25/TF-IDF).
6. FR-6 (P0) Filters and facets: category, brand, price range, rating, availability.
7. FR-7 (P0) Autocomplete/suggestions as the user types (Trie/prefix).
8. FR-8 (P1) Semantic search: embed the query and retrieve by vector similarity.
9. FR-9 (P1) Hybrid ranking: fuse lexical and semantic scores (e.g. reciprocal rank fusion).
10. FR-10 (P1) Typo tolerance and basic spell correction (edit distance / n-grams).
11. FR-11 (P2) Query understanding: synonyms, category intent, price extraction.
12. FR-12 (P2) Learning-to-rank reranking using click signals.

6.3 Recommendation

13. FR-13 (P0) Trending/popular products (time-decayed popularity).
14. FR-14 (P0) Similar products on the product page (content-based/item-item similarity).
15. FR-15 (P1) 'Users also viewed / bought' (collaborative filtering).
16. FR-16 (P1) Personalized home feed for returning users.
17. FR-17 (P1) Cold-start handling for new users and new products.
18. FR-18 (P1) Explainable recommendations ('because you viewed X', 'similar attributes').
19. FR-19 (P2) Session-based recommendations (recent session context).
20. FR-20 (P2) Diversity and novelty controls to avoid repetitive feeds.

6.4 Curation and product pages

21. FR-21 (P0) Curated ranked lists ('best picks') with a transparent composite score.
22. FR-22 (P0) Product detail page with score breakdown, attributes, similar items and recommendations.
23. FR-23 (P1) 'Best for' attribute cuts generated from product attributes.

6.5 Accounts, history and admin

24. FR-24 (P0) User accounts (sign-up/login) and session handling.
25. FR-25 (P0) Interaction logging: views, clicks, searches, wishlist, ratings.

- 26. FR-26 (P1) Wishlist and recently viewed.
- 27. FR-27 (P1) Admin dashboard: manage products, monitor search/rec quality, trigger re-index/re-train.
- 28. FR-28 (P2) Search and recommendation analytics dashboard (CTR, top queries, zero-result queries).

7. Non-Functional Requirements

Category	Requirement	Target
Performance	Search response time (p95)	<= 200 ms
Performance	Recommendation response time (p95)	<= 150 ms (served from precomputed candidates)
Performance	Page load (mobile)	LCP <= 2.5 s
Scalability	Catalogue size supported	100k+ products; 1k+ QPS design target
Availability	Uptime	99.5%+
Freshness	New interactions reflected in recommendations	Within hours (batch) / real-time for session signals
Relevance	Search nDCG@10 on a labelled set	>= 0.75
Relevance	Recommendation Precision@10	Improve over popularity baseline
Security	Auth, input validation, rate limiting, no secrets in client	OWASP basics
Privacy	Consent, minimal PII, user data export/delete	Policy + implementation
SEO	Server-rendered, indexable product/category pages	Core routes SSR
Accessibility	WCAG	2.1 AA for core flows
Observability	Logs, metrics, traces on search/recs	Dashboards + alerts

8. System Architecture

The system separates an online serving path (low-latency search and recommendations for users) from an offline/streaming path (ingestion, index building, model training and batch recommendation generation). This keeps user requests fast and lets heavy computation run in the background.

8.1 Logical components

- Client (React/Next.js): search UI, product pages, home feed, admin.
- API gateway / BFF: routing, auth, rate limiting, aggregation.
- Catalogue service: product CRUD, taxonomy, attributes.
- Search service: index + query parsing + ranking (BM25 + vector + fusion).
- Recommendation service: candidate generation + ranking + explanation.
- Interaction/event service: logs views, clicks, searches, ratings.
- User service: accounts, sessions, wishlist, history.
- Offline pipeline: ingestion, index build, embedding generation, model training, batch recs.
- Data stores: PostgreSQL (OLTP), OpenSearch/Elasticsearch or Postgres FTS (search), vector store (pgvector/Qdrant), Redis (cache), object storage (images).
- Analytics store: ClickHouse/BigQuery for events and metrics.

8.2 Online request flow

```
User -> Client -> API Gateway -> [ Search Svc | Rec Svc | Catalog Svc ]
                                   -> Redis cache (hot queries/recs)
                                   -> OpenSearch / Vector store / Postgres
                                   -> ranked JSON -> Client renders
```

8.3 Offline / batch pipeline

```
Dataset/Connector -> Ingest -> Normalize + Dedupe -> PostgreSQL
                    -> Text index build (OpenSearch) -> Search ready
                    -> Embeddings (model) -> Vector index (pgvector) -> Semantic search ready
                    -> Interaction log -> Train models (CF/ALS/content) -> Batch recs -> Redis
```

8.4 Key architectural decisions

Decision	Choice	Rationale
Serving vs compute	Precompute candidates; rank at request time	Low latency, simple scaling
Search engine	OpenSearch/Elasticsearch (or Postgres FTS for MVP)	Battle-tested IR with BM25 and facets
Vector search	pgvector (start) or Qdrant (scale)	Simple to operate; good recall
Hybrid fusion	Reciprocal Rank Fusion of lexical + vector	Robust without heavy tuning

Recommenders	Batch candidate generation + lightweight online ranking	Fits semester timeline; easy to evaluate
Caching	Redis for hot queries and recommendation sets	Protects services under load

9. Search System Design

Search is the first pillar and the clearest demonstration of information-retrieval concepts. The design moves from a simple keyword index to a hybrid semantic system, so you can ship incrementally.

9.1 Pipeline

1. Ingestion and normalization: lowercase, tokenize, remove stop words, stem/lemmatize, expand synonyms.
2. Index build: inverted index with term frequencies and document lengths; store fields for filtering/facets.
3. Query processing: parse the query, detect filters (price, brand), correct spelling, expand synonyms.
4. Lexical retrieval: score with BM25 (a TF-IDF improvement handling document length and saturation).
5. Semantic retrieval: embed query and products; retrieve top candidates by cosine similarity via ANN (HNSW).
6. Hybrid fusion: combine lexical and vector result lists with Reciprocal Rank Fusion (RRF).
7. Reranking (optional): cross-encoder or learning-to-rank using click signals.
8. Filtering and faceting: apply category/brand/price/rating filters, compute facet counts.
9. Response assembly: paginate, attach snippets, log the query for analytics.

9.2 Key concepts

Concept	What it does	Where it helps
Inverted index	Maps terms to posting lists of documents	Fast keyword retrieval
TF-IDF / BM25	Scores term importance with length normalization	Relevance ranking
Embeddings + cosine	Represents meaning as vectors	Semantic / intent matching
ANN (HNSW/IVF)	Approximate nearest-neighbour search	Fast vector retrieval at scale
Reciprocal Rank Fusion	Merges multiple ranked lists	Hybrid lexical + semantic ranking
Edit distance / n-grams	Fuzzy matching	Typo tolerance
Trie	Prefix tree	Autocomplete and suggestions

9.3 Evaluation of search

Build a small labelled query set (query -> relevant product IDs) and measure nDCG@10, MRR and MAP. Track zero-result queries and click-through rate in production.

10. Recommendation System Design

Recommendation is the second pillar. The design layers four complementary approaches so that quality improves as data grows, and degrades gracefully when data is sparse.

10.1 Signals used

Signal	Type	Weight (indicative)
Purchase	Implicit, strongest	5
Add to cart	Implicit	4
Wishlist / save	Implicit	3
Click	Implicit	2
Product view / dwell	Implicit	1
Explicit rating	Explicit	1-5 scaled

10.2 Approaches (layered)

1. Popularity / trending: time-decayed counts; always available; strong cold-start baseline.
2. Content-based: represent products by attributes (TF-IDF/embeddings); recommend by similarity to items the user engaged with.
3. Collaborative filtering (item-item): 'users who interacted with A also interacted with B' via cosine similarity on the user-item matrix.
4. Matrix factorization / ALS: learn latent user and item factors from implicit feedback for personalized ranking.
5. Hybrid: blend the above (weighted or via a learned ranker) to combine strengths.
6. Session-based (stretch): sequence models (e.g. GRU4Rec-style) for real-time session context.

10.3 Serving architecture

```
Offline (nightly):
interactions -> train item-item / ALS -> precompute top-N per user
-> store candidate lists in Redis

Online (per request):
user + context -> fetch candidates (Redis) + content similarity
-> blend + filter (already seen, out of stock) -> rank -> explain
```

10.4 Cold start

- New user: fall back to trending, category-popular and content-based on any onboarding preferences.
- New product: use content-based similarity to attach it to relevant items; expose in related items until interactions accumulate.
- Sparse data: prefer content-based and popularity; introduce CF as interactions grow.

10.5 Explainability

Every recommendation carries a reason, e.g. 'Because you viewed running shoes', 'Similar to an item in your wishlist', or 'Popular in Running Shoes this week'. This mirrors flash.co's transparency and makes the system easier to demo and defend.

10.6 Evaluation of recommendations

Metric	Meaning	Use
Precision@K	Fraction of top-K that are relevant	Ranking quality
Recall@K	Fraction of all relevant items found in top-K	Coverage of relevant items
MAP	Mean average precision	Overall ranking quality
nDCG@K	Graded relevance with position discount	Ranking with relevance levels
Hit Rate@K	Whether a relevant item appears in top-K	Practical usefulness
Coverage	Share of catalogue recommended	Avoid popularity bias
Diversity / Novelty	Variety and non-obviousness	User experience

11. Data Model

Core entities for the catalogue, users and interactions. Interaction data is the fuel for recommendations, so it is modeled explicitly and appended immutably.

Entity	Key fields	Relationships
User	id, email_hash, name, created_at	has many Interactions, Ratings, WishlistItems
Category	id, parent_id, name, slug, path	tree; has many Products
Brand	id, name, slug	has many Products
Product	id, title, description, brand_id, category_id, price, images[], attributes(jsonb), avg_rating	belongs to Brand/Category; has many Interactions
Interaction	id, user_id, product_id, type(view click cart wishlist purchase), ts, session_id	belongs to User/Product
Rating	id, user_id, product_id, value, ts	belongs to User/Product
SearchLog	id, user_id?, query, filters, result_ids[], clicked_id?, ts	analytics / LTR training
Recommendation	id, user_id, product_id, score, reason, generated_at	served from cache
CuratedList	id, title, category_id, intro, generated_at	has many CuratedListItems
CuratedListItem	list_id, product_id, rank, score	join

12. API Design

REST over HTTPS, JSON, versioned under /v1. Read endpoints are cache-friendly. Streaming is not required for v1.

12.1 Endpoints

Method & path	Purpose	Key params
GET /search	Hybrid search	q, category, brand, price_min, price_max, sort, page
GET /search/suggest	Autocomplete	q
GET /products/{id}	Product detail	-
GET /products/{id}/similar	Similar products	limit
GET /products/{id}/recommendations	Item-based recommendations	limit
GET /recommendations	Personalized feed	context, limit (auth)
GET /recommendations/trending	Trending products	category, window
GET /curated-lists	Curated lists	category
GET /curated-lists/{id}	Ranked list	-
POST /events	Log interaction	body: {product_id, type, session_id}
POST /ratings	Submit rating	body: {product_id, value}
GET /me/wishlist	Wishlist	auth
POST /auth/login	Authentication	credentials / OAuth

12.2 Sample: GET /search

```
GET /v1/search?q=running%20shoes%20flat%20feet&price_max=5000 HTTP/1.1

200 OK
{
  "query": "running shoes flat feet",
  "total": 128,
  "results": [
    { "id": "p_102", "title": "Support Runner Pro", "brand": "Stride",
      "price": 3499, "score": 0.91, "reason": "matches foot-support intent" },
    { "id": "p_884", "title": "ArchFit Trainer", "brand": "MoveOn",
      "price": 2999, "score": 0.87, "reason": "best value for flat feet" }
  ],
  "facets": { "brand": {"Stride": 12, "MoveOn": 9}, "price": {"0-2000": 40, "2000-5000":
88} }
}
```

12.3 Sample: GET /recommendations

```
GET /v1/recommendations?limit=5 HTTP/1.1
Authorization: Bearer <token>

200 OK
{
  "strategy": "hybrid",
  "items": [
    { "id": "p_102", "score": 0.94, "reason": "Because you viewed Running Shoes" },
    { "id": "p_310", "score": 0.88, "reason": "Users who viewed this also viewed" },
    { "id": "p_552", "score": 0.81, "reason": "Trending in Running Shoes" }
  ]
}
```

12.4 Sample: POST /events

```
POST /v1/events HTTP/1.1
Content-Type: application/json

{ "product_id": "p_102", "type": "click", "session_id": "s_77", "ts":
  "2026-10-05T10:12:00Z" }

202 Accepted
{ "ok": true }
```

13. Technology Stack

A practical stack that matches your full-stack skills and keeps the search/recommendation engines central.

Layer	Recommendation	Notes
Frontend	Next.js/React + TypeScript + Tailwind	SSR for SEO; product and search pages
Backend API	Node.js (NestJS/Express) or Python (FastAPI)	Python eases ML; Node shares types with frontend
ML / RecSys	Python: scikit-learn, implicit, sentence-transformers, numpy/pandas	Model training and offline batch jobs
Primary DB	PostgreSQL	Catalogue, users, interactions, curated lists
Search engine	OpenSearch/Elasticsearch (or Postgres FTS for MVP)	BM25, facets, autocomplete
Vector store	pgvector (start) or Qdrant	Semantic search and embeddings
Cache	Redis	Hot queries, recommendation sets, sessions
Object storage	S3 / Cloudflare R2	Product images
Batch / jobs	Celery / BullMQ / cron	Ingestion, embedding, training, batch recs
Analytics	ClickHouse / BigQuery (or Postgres for MVP)	Events and metrics
Auth	NextAuth / Supabase Auth / Clerk	Accounts and sessions
DevOps	Docker + GitHub Actions + Cloudflare/Vercel/Render	CI/CD and hosting
Observability	Sentry + OpenTelemetry + Grafana (or hosted)	Errors, traces, dashboards

14. Algorithms & Data Structures

This project is an excellent showcase of algorithms because search and recommendation are fundamentally algorithmic. The table maps each concept to where it is used.

DSA concept	Where used	Why
Hash map / set	Inverted index postings, dedup, caching, user-item maps	$O(1)$ average lookup
Inverted index	Keyword search	Term $\rightarrow$ documents retrieval
TF-IDF / BM25	Lexical relevance ranking	Principled term weighting
Heap / priority queue	Top-K results and recommendations	$O(n \log k)$ selection
Sorting + comparators	Ranking, multi-key sort	Ordered output
Trie	Autocomplete	$O(m)$ prefix search
Graph (DAG)	Category taxonomy, item similarity graph	Relationships and traversal
BFS / DFS	Category traversal, related-item expansion	$O(V + E)$
Union-Find	Product de-duplication during ingestion	Fast clustering
Cosine similarity	Content-based and item-item recommendations	Vector closeness
ANN (HNSW/IVF)	Semantic search and vector recs	Fast approximate nearest neighbour
Matrix factorization / ALS	Collaborative filtering	Latent factor learning
MinHash / LSH	Near-duplicate products/reviews	Scalable similarity
Edit distance / n-grams	Typo tolerance, spell correction	Fuzzy string matching
Sliding window / decay	Trending and rate limiting	Time-based aggregation
LRU cache	Hot query and rec caching	$O(1)$ eviction
Learning to rank	Final reranking from clicks	Data-driven ordering

FOR YOUR REPORT

Include the naive approach and its complexity, then the chosen algorithm and the improvement.
Example: full sort $O(n \log n)$ vs heap Top-K $O(n \log k)$; exact nearest neighbour $O(n)$ per query vs ANN $\sim O(\log n)$.

15. Development Roadmap & Milestones

A 12-week plan for a solo builder, assuming a few hours per day. Adjust to your semester length.

Week(s)	Phase	Deliverable
1	Setup & data	Repo, DB schema, dataset imported, category pages
2	Search MVP	Keyword search (BM25) with filters and facets
3	Autocomplete & relevance	Suggest API, ranking tuning, labelled query set
4	Semantic search	Embeddings + vector index + cosine retrieval
5	Hybrid search	RRF fusion + evaluation (nDCG, MAP)
6	Recs baseline	Trending + content-based similar items
7	Collaborative filtering	Item-item CF + evaluation vs baseline
8	Personalization	ALS/matrix factorization + personalized feed
9	Curation & product page	Ranked lists, score breakdown, reasons
10	Accounts & events	Auth, interaction logging, wishlist, history
11	Admin & analytics	Admin dashboard, search/rec dashboards
12	Polish & deploy	Performance, tests, deployment, report/demo

15.1 Milestones

- M1 (end wk 2): working keyword search over the full catalogue.
- M2 (end wk 5): hybrid search with measured relevance improvements.
- M3 (end wk 8): recommendation engine beating the popularity baseline.
- M4 (end wk 10): full product experience with explainable results.
- M5 (end wk 12): deployed system, metrics dashboard and report.

16. Testing, Evaluation & Metrics

16.1 Test types

Type	Covers	Tools
Unit	Scoring, similarity, parsers, fusion logic	pytest / Jest
Integration	DB + index + cache wiring	Testcontainers
API	Endpoint contracts	Postman / Schemathesis
E2E	Search and recommendation journeys	Playwright / Cypress
Load	Search and recs under concurrency	k6 / Locust
Offline ML eval	nDCG, MAP, Precision/Recall@K	Custom scripts
Online eval	CTR, dwell, conversion via A/B	Analytics store + dashboards

16.2 What success looks like

- Search nDCG@10 ≥ 0.75 and MRR improving over the keyword-only baseline.
- Recommendations beat the popularity baseline on Precision@10 and Hit Rate@K.
- p95 search latency ≤ 200 ms at the target catalogue size.
- Explainable reasons present on 100% of recommendations and curated items.
- Zero-result query rate trending down.

17. Deployment, DevOps & Operations

1. Containerize the API, search and worker services with Docker; use docker-compose for local dev.
2. Provision managed Postgres, Redis and (optionally) OpenSearch; put Cloudflare in front for CDN/WAF.
3. Deploy the web app to Vercel/Netlify and backend/workers to Render/Fly.io/AWS; separate worker pool for batch jobs.
4. CI/CD with GitHub Actions: lint, test, build, preview, deploy on merge.
5. Caching strategy: Redis for hot queries/recs; HTTP cache headers for public pages.
6. Observability: Sentry for errors, OpenTelemetry traces on search/recs, dashboards for latency, CTR and zero-result queries.
7. Scheduled jobs: nightly ingestion/index refresh, embedding updates, model retraining, batch recommendation generation.
8. Backups and a runbook for index rebuild and model rollback.

18. Risks & Mitigations

Risk	Impact	Mitigation
Sparse interaction data	Weak CF recommendations	Start with content-based + popularity; use implicit feedback; simulate data
Search relevance hard to tune	Poor user experience	Build a labelled query set early; iterate with metrics
Cold start	Bad first experience	Trending + onboarding preferences + content similarity
Scale/latency of vector search	Slow responses	ANN indexes, caching, candidate precomputation
Dataset licensing/quality	Legal/quality issues	Use permissively licensed datasets; document sources
Popularity bias	Repetitive recommendations	Diversity/novelty controls; coverage metric
Scope creep	Unfinished project	Freeze P0 scope; move extras to Future Scope

19. Future Scope

- LLM-powered features: natural-language search, review summaries and 'why this pick' narratives.
- Autonomous research agents (as in flash.co) to gather evidence and justify rankings.
- Multi-store live price comparison and affiliate monetization.
- Session-based and sequence-aware recommendations (RNN/Transformer).
- Learning-to-rank with click models; bandits for exploration.
- Multimodal search using product images.
- Real-time streaming recommendations and personalization.
- A/B testing platform for continuous relevance improvement.

20. Deliverables & Report Outline

20.1 Deliverables

- Working deployed web application (search + recommendations + curated lists).
- Source code with README, setup instructions and architecture diagram.
- Dataset documentation and ingestion scripts.
- Evaluation report with relevance and recommendation metrics.
- API documentation (OpenAPI).
- Final report and presentation/demo.

20.2 Suggested report outline

- Abstract and problem statement (choice overload, relevance, personalization).
- Introduction, objectives, scope and limitations.
- Literature review: information retrieval, BM25, embeddings, recommender systems.
- System design: architecture, DFD (0/1), ER diagram, use cases, sequence diagrams.
- Search design: indexing, ranking, hybrid fusion, algorithms and complexity.
- Recommendation design: content-based, collaborative, hybrid, cold start, explainability.
- Implementation: stack, modules, screenshots, API list.
- Evaluation and results: nDCG/MAP/Precision/Recall@K, latency, A/B or offline results.
- Deployment and operations.
- Conclusion, future scope and references.

21. Appendices

Appendix A - Dataset options

Dataset	Contents	Use
Amazon product/review datasets	Products, categories, reviews	Catalogue + implicit signals
Flipkart / Indian e-commerce datasets	Products, prices, categories	Localized catalogue
Kaggle e-commerce datasets	Varied product/order data	Quick start
MovieLens	User-item ratings	Practice CF before product data
RetailRocket / e-commerce clickstream	Views, carts, purchases	Session and implicit feedback
Self-generated synthetic interactions	Simulated views/clicks	Cold-start and testing at scale

Appendix B - Glossary

Term	Meaning
IR	Information Retrieval
BM25	Ranking function improving on TF-IDF with length normalization
Embedding	Dense vector representation of text/items
ANN	Approximate Nearest Neighbour search
HNSW	Graph-based ANN index structure
CF	Collaborative Filtering
ALS	Alternating Least Squares (matrix factorization)
RRF	Reciprocal Rank Fusion
nDCG	Normalized Discounted Cumulative Gain
MAP	Mean Average Precision
LTR	Learning to Rank
Cold start	Making recommendations without interaction history
Facet	A filterable attribute dimension (brand, price, rating)

Appendix C - Disclaimer

This is an original project concept. It is inspired by the publicly observable product patterns of flash.co but is not affiliated with, endorsed by, or produced in cooperation with Flash.co. The algorithms, architecture and roadmap are proposals intended for academic and portfolio use.